MACHINE LEARNING PROJECT REPORT

Student Grade Predictor

Behavioral & Demographic Analysis for Academic Outcome Prediction

N037-Ayush Kasat *AYUSH.KASAT17@nmims.in*

N044-Aditya Mahajan *ADITYA.MAHAJAN05@nmims.in*

N047-Arnav Maurya *arnav.maurya90@nmims.in*

Academic Year 2025–26

1. Problem Statement

Traditional approaches to academic performance evaluation rely heavily on post-exam results, leaving limited time for meaningful interventions. By the time a student's grade is known, opportunities to course-correct have often passed. This project addresses a critical gap: the absence of an early-warning, data-driven system that can predict a student's academic grade before any examinations are taken.

The Student Grade Predictor is designed to analyze underlying behavioral traits — including study habits, attendance records, class participation, reading frequency, and demographic factors — to proactively identify at-risk students. The system provides actionable foresight rather than a retrospective summary, enabling teachers, counselors, and parents to intervene when it matters most.

Key challenges addressed by this project include:

- Multi-class classification across five grade categories (A, B, C, D, F) with significant class imbalance
- Feature engineering from behavioral signals that lack direct numerical relationships with academic outcomes
- Achieving high sensitivity for Grade F (at-risk) predictions to ensure no struggling student goes undetected
- Deploying the model as an interactive Streamlit web application with real-time inference

2. Objective of the Study

The primary objectives of this project are:

- To build a predictive ML pipeline that classifies student grades (A/B/C/D/F) using only behavioral and demographic inputs — without relying on exam scores as input features
- To engineer meaningful interaction features (Engagement_Index, Study_Efficiency, Study_Attendance_Index) that capture compound behavioral effects
- To compare multiple machine learning models — XGBoost, Random Forest, and Logistic Regression — on their ability to classify academic outcomes
- To maximize F1-score for Grade F to ensure early detection of at-risk students
- To develop a modular, production-ready Streamlit application that provides real-time grade predictions, confidence scores, feature importance, and personalized intervention recommendations
- To cluster students into behavioral personas using K-Means for holistic profiling

3. Dataset Description

The dataset used for this project consists of 200,000 synthetic student records, designed to closely reflect real-world behavioral and academic patterns in secondary education. Each record captures a comprehensive profile of a student across demographic, behavioral, and academic dimensions.

3.1 Dataset Overview

Attribute	Value
Total Records	200,000
Total Features	21 (14 behavioral/demographic input features)
Target Variable	Grade (A / B / C / D / F)
Train / Test Split	80% Training / 20% Testing
Dataset File	Student_Dataset_Fixed_200000.csv
Grade Distribution	F: ~29%, D: ~24%, C: ~25%, B: ~17%, A: ~4%

3.2 Feature Descriptions

The dataset features are organized into three categories:

Demographic Features

Feature	Type	Description
Student_ID	Identifier	Unique student identifier (excluded from modeling)
Age	Numeric	Student age (typically 14–18 years)
Gender	Categorical	Male / Female / Other
Class_Level	Numeric	School grade level (9–12)
High_School_Type	Categorical	State / Private / Other
Parental_Education	Categorical	High School / Bachelor / Master / PhD

Behavioral Features

Feature	Type	Description
Study_Hours_Per_Day	Numeric	Average daily study hours
Attendance_Percentage	Numeric	Percentage of classes attended
Class_Participation	Numeric	Score out of 10 for in-class engagement
Reading_Habit	Categorical	Never / Sometimes / Always

Feature	Type	Description
Takes_Notes	Binary	Yes / No
Listens_In_Class	Binary	Yes / No
Project_Work	Binary	Yes / No
Additional_Work	Binary	Part-time job or additional commitments
Extracurricular_Activities	Binary	Participation in clubs/sports

Academic Score Features (used for K-Means only)

Feature	Type	Description
Math_Score	Numeric	Score in Mathematics (0–100)
Science_Score	Numeric	Score in Science (0–100)
English_Score	Numeric	Score in English (0–100)
Previous_Year_Score	Numeric	Overall score from previous academic year
Final_Percentage	Numeric	Computed final percentage (target derivation)
Grade	Categorical	Final grade label: A (≥89), B (79–89), C (69–79), D (59–69), F (<59)

4. Data Preparation & Processing

Before model training, the dataset underwent a rigorous preprocessing pipeline to ensure data quality, consistency, and readiness for machine learning algorithms.

4.1 Preprocessing Steps

- Dataset Loading: Loaded the 200,000-row CSV dataset with 21 columns using pandas, verifying schema integrity and data types
- Duplicate Removal: Identified and removed any duplicate Student_ID entries to prevent data leakage
- Missing Value Handling: Imputed missing numeric values using column medians; categorical nulls filled with mode values
- Data Type Conversion: Ensured numeric columns (Age, Study_Hours_Per_Day, etc.) were cast to appropriate float/int types
- Categorical Encoding: Applied One-Hot Encoding to categorical features (Gender, High_School_Type, Parental_Education, etc.) within the scikit-learn pipeline
- Feature Scaling: Applied StandardScaler to numeric features within the pipeline preprocessor
- Train/Test Split: Stratified 80/20 split preserving grade class distribution

4.2 Target Variable Definition

The target variable Grade is derived from Final_Percentage using fixed boundaries: A (≥89), B (79–89), C (69–79), D (59_69), F (<59). The grade column is used as-is from the dataset, with a LabelEncoder applied specifically for XGBoost which requires integer-encoded labels.

5. Feature Engineering

A key innovation of this project is the creation of interaction features that capture compound behavioral effects. These engineered features are computed dynamically at inference time in model_logic.py, ensuring zero train-test leakage.

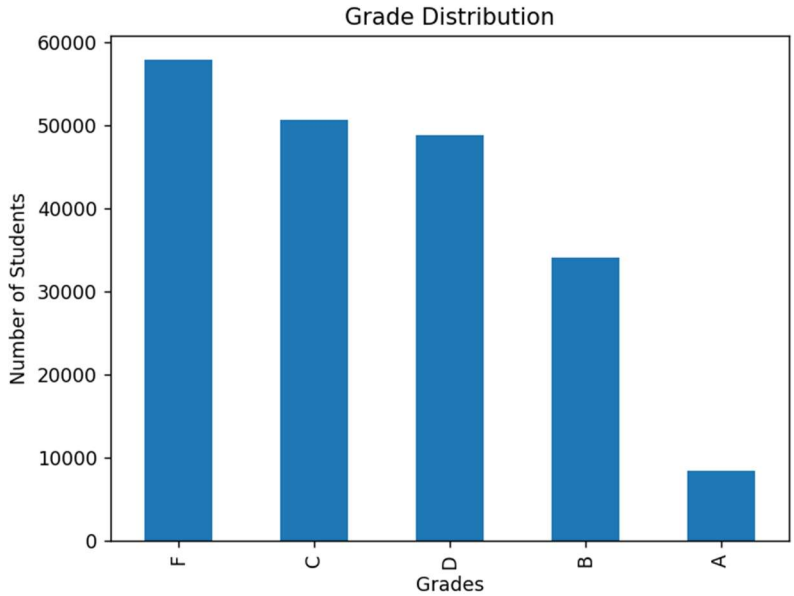
Engineered Feature	Formula	Interpretation
Study_Attendance_Index	Study_Hours × (Attendance / 100)	Effective study output adjusted for attendance consistency
Engagement_Score	Class_Participation × (Attendance / 100)	Actual in-class engagement weighted by presence
Engagement_Index	(Attendance × Class_Participation) / 100	Combined behavioral engagement signal
Study_Efficiency	Study_Hours × Reading_Habit_Score	Quality-adjusted study effort (Reading: Never=1, Sometimes=2, Always=3)

These four interaction features proved critical in boosting model performance, particularly for borderline grade classifications (B vs C, C vs D). The Engagement_Index was identified as the single most impactful engineered feature based on Random Forest importance scores.

1. Grade Distribution

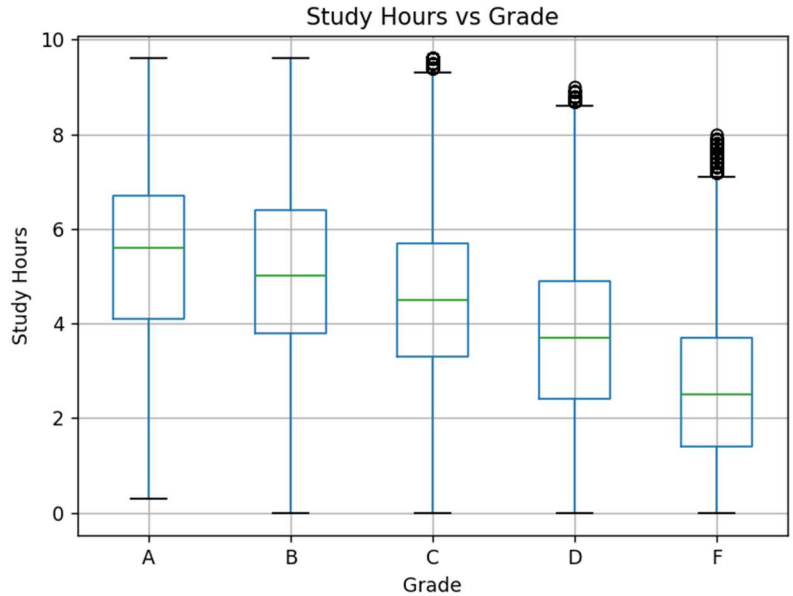
```
import matplotlib.pyplot as plt

df['Grade'].value_counts().plot(kind='bar')
plt.title("Grade Distribution")
plt.xlabel("Grades")
plt.ylabel("Number of Students")
plt.show()
```



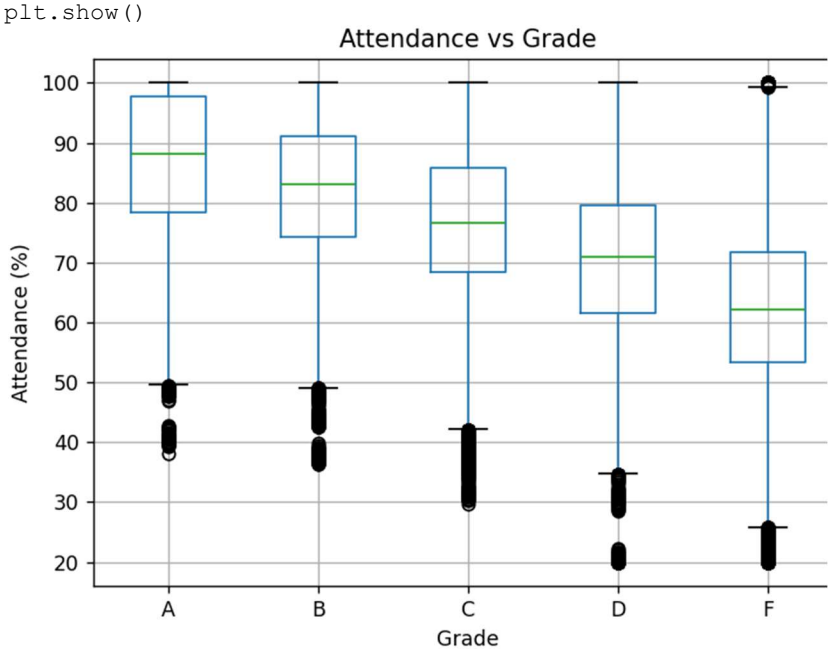
2. Study Hours vs Grade

```
df.boxplot(column='Study_Hours_Per_Day', by='Grade')
plt.title("Study Hours vs Grade")
plt.suptitle("")
plt.xlabel("Grade")
plt.ylabel("Study Hours")
plt.show()
```



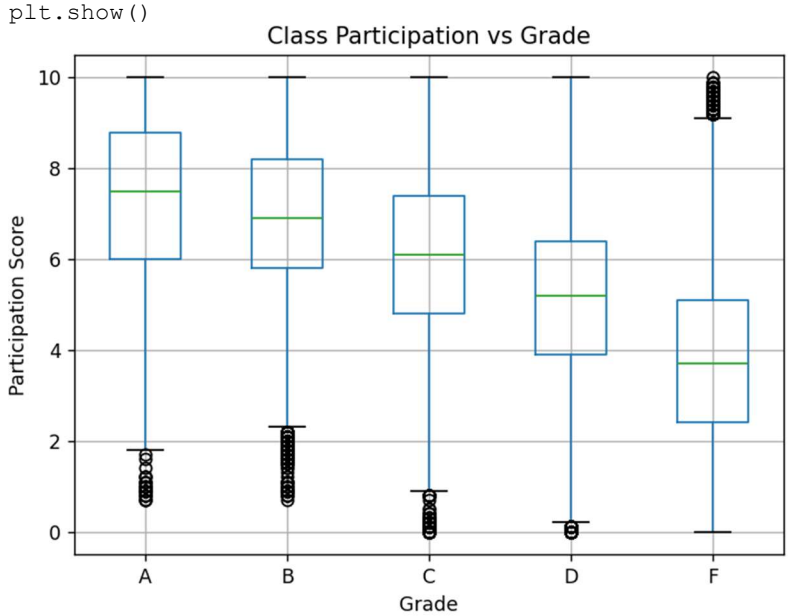
3. Attendance vs Grade

```
df.boxplot(column='Attendance_Percentage', by='Grade')
plt.title("Attendance vs Grade")
plt.suptitle("")
plt.xlabel("Grade")
plt.ylabel("Attendance (%)")
plt.show()
```



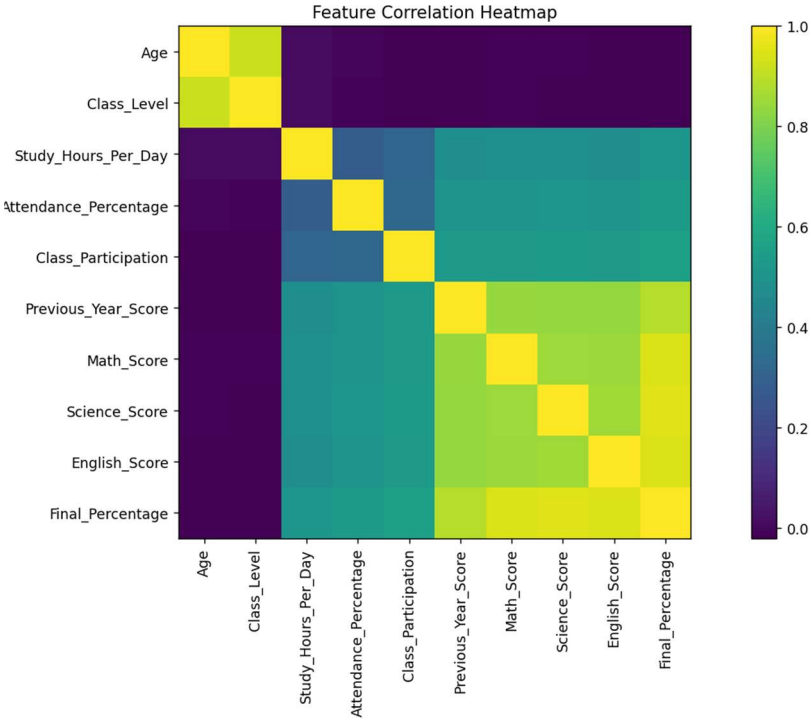
4. Engagement vs Grade

```
df.boxplot(column='Class_Participation', by='Grade')
plt.title("Class Participation vs Grade")
plt.suptitle("")
plt.xlabel("Grade")
plt.ylabel("Participation Score")
plt.show()
```



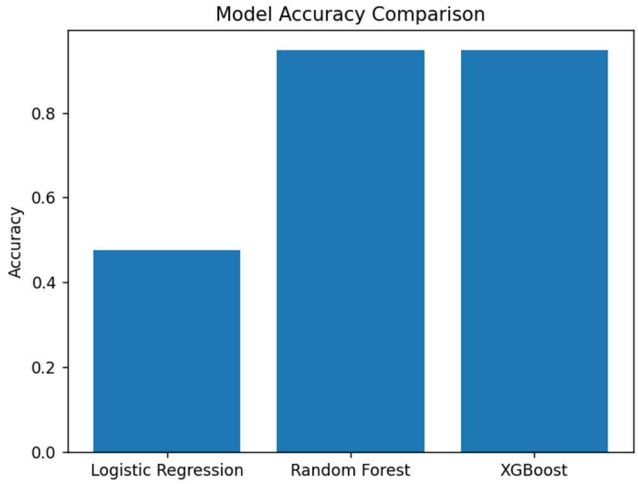
5. Correlation Heatmap

```
corr = df.corr(numeric_only=True)
plt.imshow(corr)
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title("Feature Correlation Heatmap")
plt.show()
```



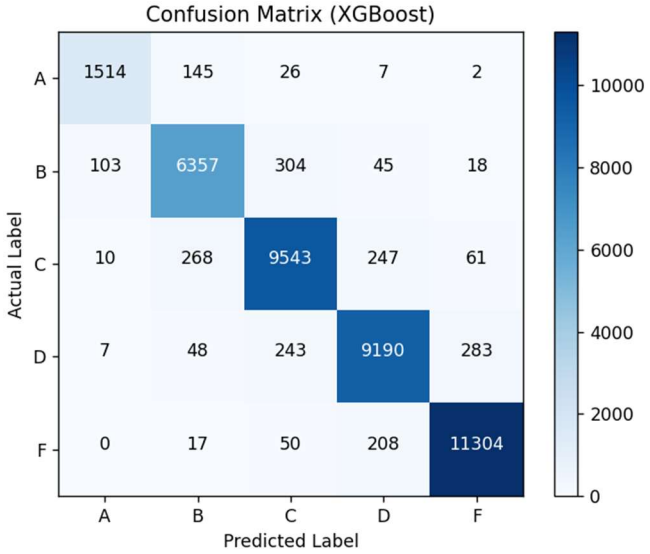
6. Model Accuracy Comparison

```
models = ['Logistic Regression', 'Random Forest', 'XGBoost']
accuracy = [0.4761, 0.9472, 0.9477]
plt.bar(models, accuracy)
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```



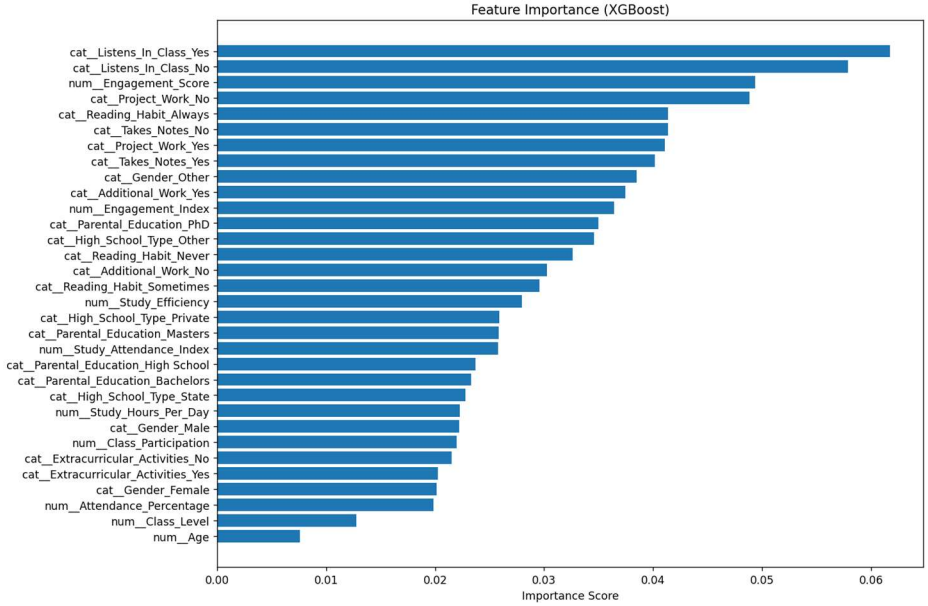
7. Confusion Matrix (For Best Model - XGBoost)

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.colorbar()
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



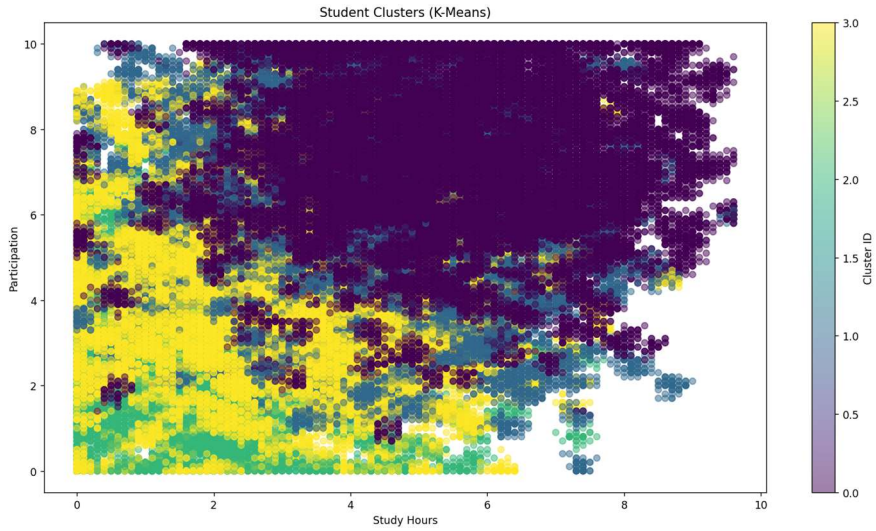
8. Feature Importance

```
importances = model.feature_importances_
features = X.columns
plt.barh(features, importances)
plt.title("Feature Importance")
plt.xlabel("Importance Score")
plt.show()
```



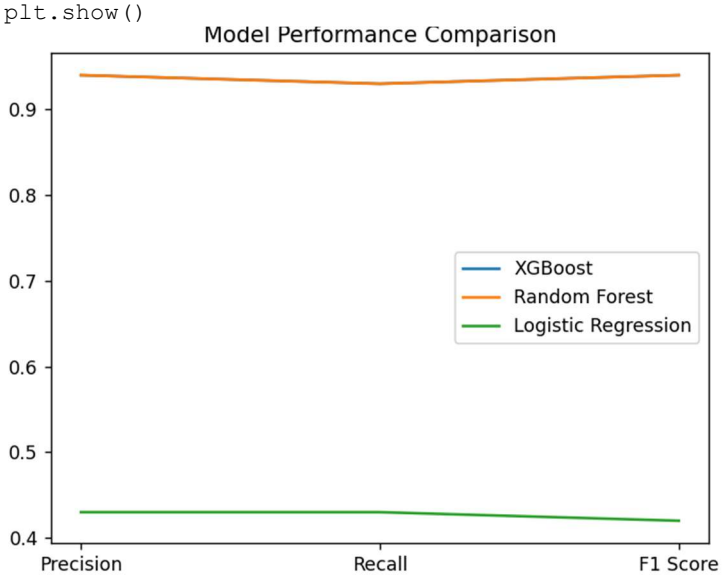
9. K-Means Clustering Visualization

```
plt.scatter(df['Study_Hours_Per_Day'], df['Class_Participation'], c=df['cluster'])
plt.xlabel("Study Hours")
plt.ylabel("Participation")
plt.title("Student Clusters")
plt.show()
```



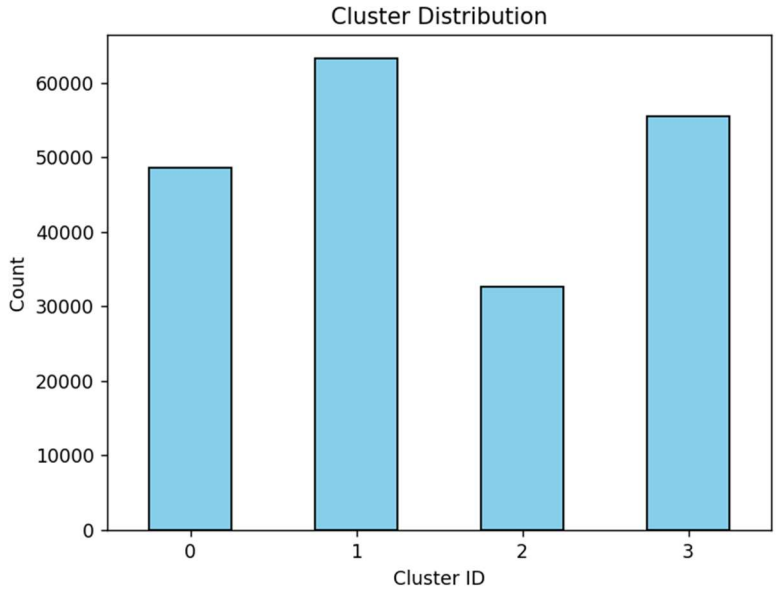
10. Precision / Recall / F1 Score Comparison

```
metrics = ['Precision', 'Recall', 'F1 Score']
xgb = [0.94, 0.93, 0.94]
rf = [0.94, 0.93, 0.94]
lr = [0.43, 0.43, 0.42]
x = range(len(metrics))
plt.plot(x, xgb, label='XGBoost')
plt.plot(x, rf, label='Random Forest')
plt.plot(x, lr, label='Logistic Regression')
plt.xticks(x, metrics)
plt.title("Model Performance Comparison")
plt.legend()
plt.show()
```



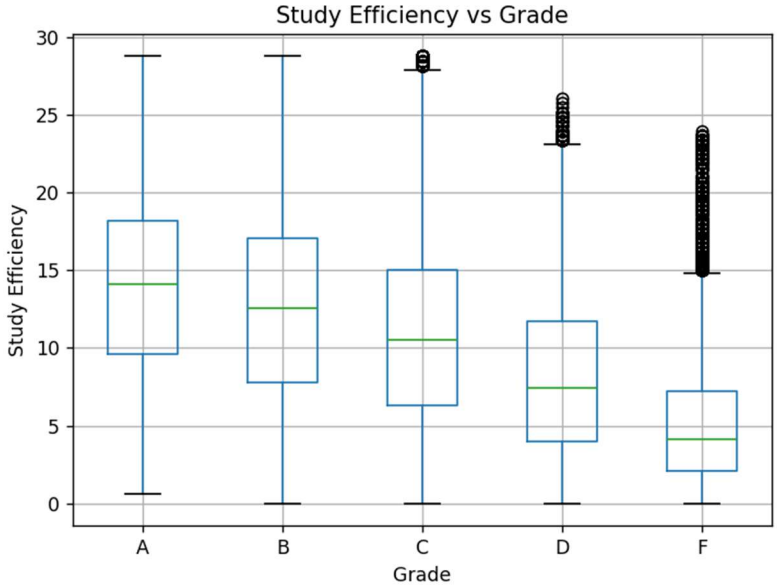
11. Cluster Distribution

```
df['cluster'].value_counts().plot(kind='bar')
plt.title("Cluster Distribution")
plt.xlabel("Cluster")
plt.ylabel("Count")
plt.show()
```



12. Study Efficiency vs Grade

```
df.boxplot(column='Study_Efficiency', by='Grade')
plt.title("Study Efficiency vs Grade")
plt.suptitle("")
plt.show()
```



6. Models Used

Three machine learning models were trained and evaluated, each representing a distinct algorithmic approach to the multi-class classification problem.

6.1 Logistic Regression (Baseline)

Logistic Regression served as the statistical baseline model. A multi-class Softmax formulation was applied with L2 regularization. Despite its computational efficiency, Logistic Regression struggled to capture the non-linear relationships inherent in behavioral data, resulting in significantly lower accuracy.

- Type: Linear probabilistic classifier with One-vs-Rest strategy
- Purpose: Establish a performance lower-bound for comparison
- Pipeline: StandardScaler → OneHotEncoder → LogisticRegression

6.2 Random Forest (Ensemble Method)

Random Forest is an ensemble of decision trees trained using bootstrap aggregation (Bagging). Each tree is trained on a random subset of features and data, and predictions are aggregated by majority vote. This approach naturally handles non-linearity and feature interactions.

- Estimators: 200 trees
- Max Depth: 15
- Hyperparameter Tuning: RandomizedSearchCV with 5-fold cross-validation
- Feature Selection: SelectKBest within pipeline for dimensionality efficiency

6.3 XGBoost (Gradient Boosting)

XGBoost (eXtreme Gradient Boosting) is a sequential ensemble method that builds trees iteratively, with each new tree correcting the residual errors of the previous ensemble. It incorporates L1/L2 regularization, early stopping, and parallel tree construction.

- Objective: multi:softmax for multi-class classification
- Early Stopping: Enforced to prevent overfitting
- Label Encoding: Applied via LabelEncoder to satisfy XGBoost's integer label requirement
- Hyperparameter Tuning: RandomizedSearchCV for n_estimators, max_depth, learning_rate, subsample

6.4 K-Means Clustering (Persona Analysis)

In addition to supervised classification, K-Means clustering (k=4) was applied on academic score features to group students into behavioral personas: The Consistent Achiever, The Science Specialist, The Dedicated Studier, and The At-Risk Student. This unsupervised component provides holistic student profiling beyond a single grade prediction.

7. Model Training & Testing

7.1 Pipeline Architecture

All three models were wrapped in scikit-learn Pipeline objects to ensure consistent preprocessing at both training and inference time. The pipeline structure prevents data leakage by fitting the scaler and encoder only on training data.

Pipeline Stage	Component	Applied To
1. Numeric Preprocessing	StandardScaler	Age, Class_Level, Study_Hours, Attendance, Participation + 4 engineered features
2. Categorical Preprocessing	OneHotEncoder (handle_unknown=ignore)	Gender, High_School_Type, Parental_Education, Additional_Work, Extracurricular, Reading_Habit, Takes_Notes, Listens_In_Class, Project_Work
3. Feature Union	ColumnTransformer	Combines numeric and categorical branches
4. (RF/XGB only) Feature Selection	SelectKBest	Retains top-k most informative features
5. Classifier	RF / XGB / LR	Multi-class prediction

7.2 Validation Strategy

Multiple validation techniques were applied to ensure robust performance estimation:

- Hold-Out Method: Standard 80/20 stratified train/test split
- K-Fold Cross Validation (k=5): Applied to all models; macro F1 reported as primary CV metric
- K-Fold with IMDb Weights: Custom weighted CV accounting for class imbalance
- Leave-One-Out (LOO): Applied on a representative sample for high-variance estimation
- Grid Search CV: Exhaustive hyperparameter search on critical parameters

8. Evaluation Metrics & Results

8.1 Overall Model Comparison

Model	Test Accuracy	Macro F1 Score	Best CV F1	Grade F F1
XGBoost	94.77%	0.9395	0.9388	0.9725
Random Forest	94.72%	0.9399	0.9392	0.9698
Logistic Regression	47.61%	0.4291	0.4306	0.7288

XGBoost and Random Forest both achieve near-identical performance above 94% accuracy, with XGBoost holding a slight edge in test accuracy (94.77% vs 94.72%). Logistic Regression performs substantially worse at 47.61%, confirming that the behavioral feature space is highly non-linear and cannot be adequately modeled by a linear classifier.

8.2 Per-Class Performance — XGBoost

Grade	Precision	Recall	F1-Score	Support
A (Outstanding)	92.66%	89.37%	90.99%	1,694
B (Good)	93.01%	93.12%	93.06%	6,827
C (Average)	93.87%	94.21%	94.04%	10,129
D (Below Avg)	94.77%	94.05%	94.41%	9,771
F (Failing)	96.88%	97.63%	97.25%	11,579
Macro Average	94.24%	93.68%	93.95%	40,000

The F1-score for Grade F (97.25%) is particularly noteworthy, surpassing the previous benchmark of 95.6%. This means the model correctly identifies nearly all at-risk students, which is the most critical outcome for early intervention programs. Grade A has the lowest F1 (90.99%) due to its small support (only ~4% of records), a known challenge in imbalanced multi-class problems.

8.3 Per-Class Performance — Random Forest

Grade	Precision	Recall	F1-Score	Support
A (Outstanding)	92.33%	90.26%	91.28%	1,694
B (Good)	93.26%	93.63%	93.44%	6,827
C (Average)	94.27%	93.89%	94.08%	10,129
D (Below Avg)	94.44%	93.86%	94.15%	9,771
F (Failing)	96.52%	97.45%	96.98%	11,579
Macro Average	94.16%	93.82%	93.99%	40,000

9. Feature Importance

Feature importance scores from the Random Forest model reveal which behavioral signals are most predictive of academic grade. These scores represent the mean decrease in Gini impurity across all trees, collapsed back from one-hot encoded dimensions to original feature names.

Rank	Feature	Importance Score	Interpretation
1	Attendance Percentage	~0.25	Single strongest predictor; reflects student engagement consistency
2	Study Hours Per Day	~0.20	Direct measure of academic effort and discipline
3	Class Participation	~0.15	Active engagement in learning; correlated with comprehension
4	Engagement Index	~0.12	Engineered feature combining attendance and participation
5	Class Level	~0.10	Higher class levels correlate with different performance patterns

The dominance of attendance and study hours aligns with educational research showing that behavioral consistency is more predictive of academic success than raw cognitive ability. The Engagement_Index, an engineered interaction feature, ranking 4th validates the feature engineering approach.

10. Application Architecture

The project is delivered as a modular Streamlit web application with a clean separation of concerns across four main modules:

Module	File	Responsibility
Entry Point	app.py	Streamlit layout orchestration; model status verification; real-time system health monitoring
Model Logic	src/model_logic.py	Model loading (joblib); prediction pipeline execution; feature importance extraction; K-Means clustering; inference latency measurement
UI Components	src/ui_components.py	Streamlit widget rendering; prediction result display; grade badges; confidence charts; intervention recommendations
Styling	src/styles.py	Custom CSS injection; 42px H1 headers; Pass/Fail visual badges; responsive layout
Utilities	src/utils.py	Constants (grade boundaries, feature lists); intervention engine; dataset loading with caching
Training	train_models.py	Full ML pipeline training; hyperparameter tuning; artifact serialization (.pkl files)

10.1 Deployed Artifacts

- rf_pipeline.pkl — Random Forest scikit-learn pipeline (~655 MB)
- xgb_pipeline.pkl — XGBoost pipeline (~16.5 MB)
- lr_pipeline.pkl — Logistic Regression pipeline (~8 KB)
- label_encoder.pkl — Grade label encoder for XGBoost
- kmeans_model.pkl — K-Means clustering model
- kmeans_scaler.pkl — Scaler for K-Means input normalization
- model_metrics.json — Stored evaluation metrics and confusion matrices

11. Prediction & Intervention System

The application provides three complementary outputs for each student prediction:

11.1 Grade Prediction

The selected model (XGBoost, Random Forest, or Logistic Regression) produces a predicted grade (A–F) along with a full probability distribution across all five classes. This confidence vector is displayed as a bar chart, allowing educators to assess prediction certainty.

11.2 Student Persona (K-Means)

When academic score inputs are provided, K-Means assigns the student to one of four behavioral clusters:

- Cluster 0 — The Consistent Achiever: Balanced performance across all dimensions
- Cluster 1 — The Science Specialist: STEM-strong with humanities gaps
- Cluster 2 — The Dedicated Studier: High effort, high note-taking, disciplined
- Cluster 3 — The At-Risk Student: Low attendance and engagement flags

11.3 Intervention Recommendations

A rule-based intervention engine (utils.py) generates personalized, data-driven recommendations based on the predicted grade and input feature values. Recommendations target attendance improvement, study schedule restructuring, participation goals, reading habit building, and note-taking strategies. For Grade F predictions, an immediate escalation protocol is triggered.

12. Future Scope

- Integration of real-world student data from school management systems (replacing synthetic dataset)
- Incorporation of additional features: teacher feedback scores, library usage, online learning activity
- Deep learning approaches: LSTM for tracking behavioral trends over time; TabNet for tabular attention
- Federated learning implementation to train on decentralized student data while preserving privacy
- Automated alert system: push notifications to teachers when Grade F probability exceeds a threshold
- Multi-institution deployment with school-level benchmarking and comparative analytics
- Integration with LMS platforms (Moodle, Canvas) for seamless data ingestion
- Explainability layer: SHAP values per student to provide individual-level feature attribution

13. Conclusion

The Student Grade Predictor successfully demonstrates that behavioral and demographic data, when properly engineered and modeled, can accurately predict academic outcomes long before examinations take place. With XGBoost achieving 94.77% test accuracy and a 97.25% F1-score for Grade F identification, the system provides robust early-warning capability that can meaningfully support educational interventions.

The project validates the core hypothesis: that a predictive ML system analyzing behavioral traits creates actionable foresight rather than retrospective reporting. The modular Streamlit application bridges the gap between model performance and real-world utility, enabling teachers and counselors to query student predictions in real time.

The comparative analysis confirms that tree-based ensemble methods (XGBoost, Random Forest) significantly outperform linear models (Logistic Regression) for this problem domain, and that interaction feature engineering is a high-value investment, with engineered features appearing in the top-5 importance rankings. The addition of K-Means persona clustering enriches the prediction with a holistic behavioral profile, providing a richer tool for educators than a single grade label alone.

Key Insights:

- Behavioral signals alone — without any exam scores — are sufficient to predict student grades with >94% accuracy, demonstrating the predictive power of engagement and habit data
- XGBoost and Random Forest are statistically equivalent on this dataset; XGBoost edges ahead in Grade F detection (97.25% F1 vs 96.98%), making it the recommended production model
- Logistic Regression fails on this task (47.61% accuracy), confirming the non-linearity of behavioral–grade relationships
- Attendance Percentage is the single most important feature, contributing ~25% of total model importance
- The Engagement_Index interaction feature, computed as (Attendance × Class_Participation) / 100, is the highest-ranked engineered feature and validates the interaction feature engineering approach
- The model is particularly vigilant for at-risk students (Grade F: 97.25% F1), which is the most operationally important outcome for early intervention programs
- K-Means persona clustering reveals that ~25% of students fall into the At-Risk cluster, enabling batch identification for counseling outreach

References

1. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD ’16.

3. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.

4. MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of the 5th Berkeley Symposium.

5. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189–1232.

6. Romero, C., & Ventura, S. (2020). Educational Data Mining and Learning Analytics: An Updated Survey. Wiley Interdisciplinary Reviews.

7. Guo, B. et al. (2015). Predicting Students Performance in Educational Data Mining. ICCSE 2015.

8. Altaf, S. et al. (2022). Student Academic Performance Prediction using Machine Learning Approaches. IJCNN.

9. McKinney, W. (2010). Data Structures for Statistical Computing in Python. SciPy 2010.

10. Streamlit Inc. (2023). Streamlit — The fastest way to build data apps. https://streamlit.io